

Proyecto Opcional — IC7602 Redes (2026-01)

Fecha de entrega: 19 de marzo de 2026

Archivo: `Documentacion.md`

Integrantes:

- Mariela Solano Gómez
- Alejandra Delgado Pérez
- Joshua Obando Castro
- Roilin Navarro Vargas

Tabla de contenidos

- Proyecto Opcional — IC7602 Redes (2026-01)
 - Tabla de contenidos
 - Introducción
 - Arquitectura requerida
 - Namespaces
 - Flujo esperado
 - Despliegue automatizado completo
 - Requisitos del entorno
 - Requisitos mínimos
 - Herramientas
 - Verificación rápida (PowerShell)
 - Estructura del repositorio
 - Instalación y ejecución paso a paso
 - Pruebas realizadas
 - Prueba 0 — Estado general
 - Prueba 1 — Apache1 (port-forward)
 - Prueba 2 — Apache2 (port-forward)
 - Prueba 3 — Ingress por rutas (si el entorno lo permite)
 - Prueba 4 — Router, se hace dentro del cluster.
 - Prueba 5 — Router: verificación técnica (iptables)
 - Problemas encontrados y solución
 - Recomendaciones
 - Conclusiones (mínimo 10)

Introducción

Este proyecto implementa una arquitectura en **Kubernetes** con dos namespaces (`publico` y `privado`) para separar servicios internos y componentes de entrada y salida.

Componentes implementados:

- **Apache1 y Apache2** en `privado` (servicios web internos, Service `ClusterIP`).
- **Ingress Controller NGINX** en `publico` para enrutar por rutas HTTP (`/Apache1`, `/Apache2`).

- **Router (Ubuntu + iptables)** en **publico** para redirección por puertos:
 - **8080** → Apache1
 - **8081** → Apache2
 - **80** → Ingress Controller
 - **5601** TCP/UDP → Asterisk

Arquitectura requerida

Namespaces

- **privado**: Apache1, Apache2 (y Asterisk/FreePBX cuando se implemente).
- **publico**: Ingress Controller + Router.

Flujo esperado

1) Acceso web centralizado (HTTP)

- Cliente → **Router:80** → **Ingress Controller** → Apache1 o Apache2
- Rutas:
 - **http://<entrada>/Apache1** → Apache1
 - **http://<entrada>/Apache2** → Apache2

2) Acceso directo por puertos

- Cliente → **Router:8080** → Apache1
- Cliente → **Router:8081** → Apache2

3) Telefonía

- Cliente → **Router:5601 TCP UDP**

Despliegue automatizado completo

```
helm install proyecto ./charts/proyecto-redes
```

Este comando realiza automáticamente:

- Creación de namespaces (**publico** y **privado**)
- Despliegue de servicios internos (Apache1, Apache2, Asterisk)
- Despliegue del router con reglas NAT
- Instalación del Ingress Controller (NGINX)
- Configuración del recurso Ingress
- Conectividad entre todos los componentes mediante DNS interno

Requisitos del entorno

Enfocado en **Windows 10 u 11** con **Docker Desktop + WSL2**.

Requisitos mínimos

- Windows 10/11 (64 bits)
- Virtualización habilitada (BIOS)
- 8 GB RAM mínimo (16 GB recomendado)
- Conexión a internet

Herramientas

- Docker Desktop (con WSL2)
- Kubernetes habilitado en Docker Desktop
- kubectl
- helm
- git

Verificación rápida (PowerShell)

```
kubectl config use-context docker-desktop
kubectl get nodes
helm version
docker version
git --version
```

Estructura del repositorio

```
proyectos/ProyectoOpcional/
├── k8s/
│   ├── namespaces.yaml
│   └── ingress.yaml
├── charts/
│   ├── apache1/
│   │   ├── app/ (Dockerfile + index.html)
│   │   ├── templates/
│   │   ├── Chart.yaml
│   │   └── values.yaml
│   ├── apache2/
│   │   ├── app/ (Dockerfile + index.html)
│   │   ├── templates/
│   │   ├── Chart.yaml
│   │   └── values.yaml
│   └── router/
│       ├── app/ (Dockerfile + router.sh)
│       ├── templates/
│       ├── Chart.yaml
│       └── values.yaml
```

Instalación y ejecución paso a paso

Ejecutar los comandos desde: **proyectos/ProyectoOpcional**

1. Confirmar que Kubernetes esté activo

```
kubectl config use-context docker-desktop
kubectl get nodes
Esperado: 1 nodo Ready.
```

2. Preparación del entorno

Antes de desplegar el sistema, es necesario construir las imágenes Docker de los servicios:

```
docker build -t apache1-custom:latest ./charts/apache1/app
docker build -t apache2-custom:latest ./charts/apache2/app
docker build -t router-custom:latest ./charts/router/app
```

3. Despliegue automatizado con Helm

El despliegue completo del sistema se realiza mediante los siguientes comandos:

```
helm dependency build ./charts/proyecto-redes
helm install proyecto ./charts/proyecto-redes
```

Pruebas realizadas

Nota importante (Windows + Docker Desktop + kind):

A veces los NodePorts no son accesibles desde Windows por temas de red o WSL o Docker.

Por eso, las pruebas más reproducibles son **dentro del cluster**

Prueba 0 — Estado general

```
kubectl get pods -A
kubectl get svc -A
kubectl get ingress -A
```

```
PS C:\Users\marie\Desktop\I-S-2026\Redes\2026-01-2022437963-IC7602\proyectos\ProyectoOpcional> kubectl get pods -A
NAMESPACE   NAME                                     READY   STATUS    RESTARTS   AGE
kube-system  coredns-66bc5c9577-jlbpz              1/1     Running   1          24h
kube-system  coredns-66bc5c9577-m726z              1/1     Running   1          24h
kube-system  etcd-docker-desktop                   1/1     Running   1          24h
kube-system  kube-apiserver-docker-desktop         1/1     Running   1          24h
kube-system  kube-controller-manager-docker-desktop 1/1     Running   1          24h
kube-system  kube-proxy-fl7fk                      1/1     Running   1          24h
kube-system  kube-scheduler-docker-desktop         1/1     Running   1          24h
kube-system  storage-provisioner                   1/1     Running   1          24h
kube-system  vpnkit-controller                     1/1     Running   1          24h
privado      asterisk-bb7d8854b-dkpjq              1/1     Running   0          8m22s
privado      proyecto-apache1-7f9b7bfd9c-mq98d     1/1     Running   0          8m22s
privado      proyecto-apache2-5d5968bdf9-5nhrh     1/1     Running   0          8m22s
publico      proyecto-ingress-nginx-controller-76cb7f4cbb-kdqz7 1/1     Running   0          8m22s
publico      proyecto-router-5d79676cd8-4r49d      1/1     Running   4 (7m34s ago) 8m22s
PS C:\Users\marie\Desktop\I-S-2026\Redes\2026-01-2022437963-IC7602\proyectos\ProyectoOpcional>
```

```
PS C:\Users\marie\Desktop\I-S-2026\Redes\2026-01-2022437963-IC7602\proyectos\ProyectoOpcional> kubectl get svc -A
```

NAMESPACE	NAME	AGE	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
default	kubernetes	24h	ClusterIP	10.96.0.1	<none>	443/TCP
kube-system	kube-dns	24h	ClusterIP	10.96.0.10	<none>	53/UDP,53/TCP,9153/TCP
privado	asterisk	9m51s	ClusterIP	10.111.222.24	<none>	5060/TCP,5060/UDP
privado	proyecto-apache1	9m51s	ClusterIP	10.96.4.116	<none>	80/TCP
privado	proyecto-apache2	9m51s	ClusterIP	10.97.216.33	<none>	80/TCP
publico	proyecto-ingress-nginx-controller	9m51s	LoadBalancer	10.109.115.60	localhost	80:32059/TCP,443:30734/TCP
publico	proyecto-router	9m51s	NodePort	10.101.239.133	<none>	80:32003/TCP,8080:32360/TCP,8081:32146/TCP,5601:30136/UDP

```
PS C:\Users\marie\Desktop\I-S-2026\Redes\2026-01-2022437963-IC7602\proyectos\ProyectoOpcional> kubectl get ingress -A
```

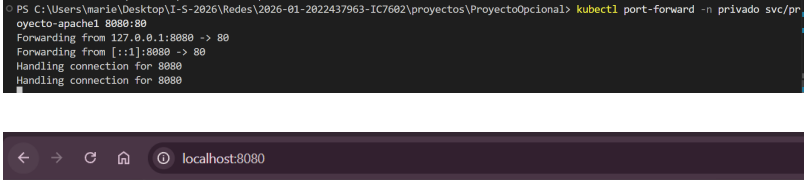
NAMESPACE	NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
privado	apache-ingress	nginx	*	localhost	80	8m45s

Prueba 1 — Apache1 (port-forward)

```
kubect1 port-forward -n privado svc/proyecto-apache1 8080:80
```

Abrir: http://localhost:8080

Esperado: Hola Apache1

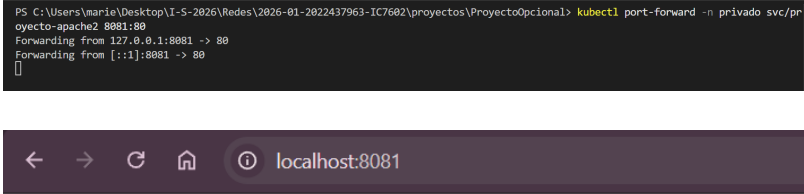


Prueba 2 — Apache2 (port-forward)

```
kubect1 port-forward -n privado svc/proyecto-apache2 8081:80
```

Abrir: http://localhost:8081

Esperado: Hola Apache2



Prueba 3 — Ingress por rutas (si el entorno lo permite)

Probar:

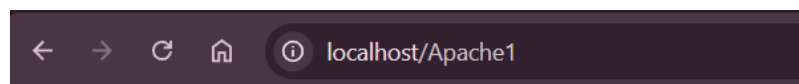
`http://localhost/Apache1`

`http://localhost/Apache2`

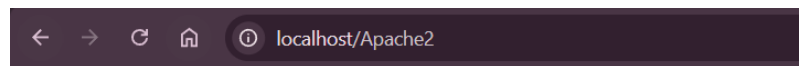
Esperado:

`/Apache1 → Hola Apache1`

`/Apache2 → Hola Apache2`



Hola Apache1



Hola Apache2

Si no funciona directo por localhost, usar Prueba 4, la cual se hace dentro del cluster, que es la más confiable.

Prueba 4 — Router, se hace dentro del cluster.

- 4.1 Crear un pod temporal con curl

```
kubectl run -it --rm test-curl --image=curlimages/curl -n publico -- sh
```

- 4.2 Probar redirecciones del router

- Router → Apache1

```
curl -v http://router.publico.svc.cluster.local:8080/

- Router → Apache2

curl -v http://router.publico.svc.cluster.local:8081/

- Router → Ingress → Apache1

curl -v http://router.publico.svc.cluster.local:80/Apache1

- Router → Ingress → Apache2

curl -v http://router.publico.svc.cluster.local:80/Apache2

Esperado: HTTP 200 + Hola Apache1/2

Y para salir:

exit
```

Prueba 5 — Router: verificación técnica (iptables)

```
kubectl exec -it -n publico deployment/proyecto-router -- bash

Dentro:

cat /proc/sys/net/ipv4/ip_forward

iptables -t nat -L PREROUTING -n -v
iptables -t nat -L OUTPUT -n -v
iptables -t nat -L POSTROUTING -n -v

exit

Esperado: reglas DNAT + MASQUERADE e ip_forward = 1.
```

```
PS C:\Users\maria\Desktop\1-5-2026\Redes\0206-01-2026\37963-1C\680\proyectos\ProyectoOptional> kubectl exec -it -n publico deployment/proyecto-router -- bash
root@proyecto-router-5d79676cd8-4r49d:/# cat /proc/sys/net/ipv4/ip_forward
1
root@proyecto-router-5d79676cd8-4r49d:/# iptables -t nat -L PREROUTING -n -v
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination
0 0 DNAT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:8080 to:10.96.4.116:80
0 0 DNAT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:8081 to:10.97.216.33:80
0 0 DNAT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:80 to:10.109.115.60:80
0 0 DNAT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:5601 to:10.111.222.24:5060
0 0 DNAT udp -- * * 0.0.0.0/0 0.0.0.0/0 udp dpt:5601 to:10.111.222.24:5060
root@proyecto-router-5d79676cd8-4r49d:/# iptables -t nat -L OUTPUT -n -v
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination
0 0 DNAT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:8080 to:10.96.4.116:80
0 0 DNAT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:8081 to:10.97.216.33:80
0 0 DNAT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:80 to:10.109.115.60:80
0 0 DNAT tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:5601 to:10.111.222.24:5060
0 0 DNAT udp -- * * 0.0.0.0/0 0.0.0.0/0 udp dpt:5601 to:10.111.222.24:5060
root@proyecto-router-5d79676cd8-4r49d:/# iptables -t nat -L POSTROUTING -n -v
Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target prot opt in out source destination
0 0 MASQUERADE tcp -- * eth0 0.0.0.0/0 10.96.4.116 tcp dpt:80
0 0 MASQUERADE tcp -- * eth0 0.0.0.0/0 10.97.216.33 tcp dpt:80
0 0 MASQUERADE tcp -- * eth0 0.0.0.0/0 10.109.115.60 tcp dpt:80
0 0 MASQUERADE tcp -- * eth0 0.0.0.0/0 10.111.222.24 tcp dpt:5060
0 0 MASQUERADE udp -- * eth0 0.0.0.0/0 10.111.222.24 udp dpt:5060
0 0 MASQUERADE all -- * eth0 0.0.0.0/0 0.0.0.0/0
root@proyecto-router-5d79676cd8-4r49d:/#
```

Problemas encontrados y solución

- Problema 1: Ingress en namespace incorrecto.
 - Que pasaba: `services "apache1" not found`
 - Causa: Ingress creado en `publico`, pero Apache está en `privado`
 - Solución:** El Ingress debe estar en `privado`.
- Problema 2: "Not Found" al usar `/Apache1` o `/Apache2`
 - Causa: Apache recibe la ruta completa `/Apache1`, pero el sitio está en `/`
 - Solución: usar rewrite con:
 - `nginx.ingress.kubernetes.io/use-regex`
 - `nginx.ingress.kubernetes.io/rewrite-target`
- Problema 3: NodePort inaccesible desde Windows (Docker Desktop + kind)
 - Que pasaba: `curl http://127.0.0.1:<nodePort>` falla
 - Causa: aislamiento de red Windows → WSL2 → Docker Desktop → red interna kind
 - Solución: pruebas dentro del cluster (Prueba 4) y evidencias con `kubectl describe svc`.

Recomendaciones

1. Hacer un "Quickstart" corto y luego el detalle, para que el profe lo ejecute rápido.
2. Probar Apache (interno) antes de Ingress y Router, así se depura por capas.
3. Usar `helm upgrade --install` para reinstalar sin pelearse con "ya existe".
4. No ejecutar `helm create` sobre charts ya modificados (puede sobrecribir y perder cambios).
5. Mantener `imagePullPolicy: IfNotPresent` cuando se usan imágenes locales.
6. Siempre validar con `kubectl get`, `kubectl describe` y `kubectl logs`.
7. Separar componentes en namespaces mejora orden y seguridad.
8. Documentar el "por qué" de decisiones (rewrite, pruebas dentro del cluster).
9. Para debug, usar `kubectl exec + iptables` + un pod temporal con `curl`.
10. Tener comandos de limpieza para reiniciar el ambiente si algo se rompe.
11. Usar pruebas reproducibles con "resultado esperado" claro.
12. Guardar evidencias (capturas/salidas) de las pruebas principales.

Conclusiones (mínimo 10)

1. Kubernetes permite separar componentes claramente usando namespaces.
2. `ClusterIP` es ideal para servicios internos (Apache en `privado`).
3. Ingress Controller resuelve enrutamiento HTTP por rutas (L7).
4. El rewrite es necesario cuando el backend no espera prefijos como `/Apache1`.
5. Un router con iptables permite redirecciones por puertos (L4) sin depender de HTTP.
6. iptables requiere permisos especiales (`NET_ADMIN/privileged`) dentro del contenedor.
7. Docker Desktop + kind en Windows puede limitar acceso a NodePort desde el host.
8. Probar desde dentro del cluster es una forma consistente y reproducible de validar redirecciones.
9. Helm ayuda a automatizar despliegues y reinstalaciones de manera ordenada.
10. La documentación y pruebas reproducibles son clave para la evaluación, no solo el código.
11. La arquitectura es ampliable: se pueden sumar más servicios sin rehacer todo.
12. Entender PREROUTING/OUTPUT/POSTROUTING facilita depurar problemas de NAT.